

Inhaltsverzeichnis der Nodges-Dokumentation

Dieses Dokument bietet eine tiefgreifende, strukturierte Übersicht über die gesamte technische und konzeptionelle Dokumentation des Projekts "Nodges" (3D Network Visualization with Three.js). Die Dokumentation spiegelt die Philosophie, die 3D-Engine und die agentischen Workflows (ab Build 5 bis Build 10) in höchstem Detailgrad wider.

- [01 Philosophie, Kognition und Visuelle Kommunikation]
(0_102_10_01_Philosophie_und_Kognition.md)
- [02 Vision und Systemarchitektur](0_102_10_02_Systemarchitektur.md)
- [03 LLM-Integration und Datenpipelines](0_102_10_03_Datenpipelines_und_LLM.md)
- [04 Datenmodell und Validierung](0_102_10_04_Datenmodell_und_Validierung.md)
- [05 3D-Rendering und Szenen-Management]
(0_102_10_05_Visual_Mapping_und_Szenen_Management.md)
- [06 Benutzeroberfläche und Interaktion]
(0_102_10_06_Benutzeroberflaeche_und_Interaktion.md)
- [07 Algorithmen, Layout Engine und Physik](0_102_10_07_Layout_Engine_und_Physik.md)
- [08 Visuelle Effekte und Feedback-Systeme]
(0_102_10_08_Visuelle_Effekte_und_Feedback.md)
- [09 Interaktions-Design und Input-Processing]
(0_102_10_09_Interaktions_Design_und_Input_Processing.md)
- [10 Entwicklungs-Guide, Testing und Deployment]
(0_102_10_10_Entwicklung_und_Deployment.md)
- [11 Historie und Entscheidungen (ADRs)](0_102_10_11_Historie_und_Entscheidungen.md)

- [12 Quickstart und Tutorial](0_102_10_12_Quickstart_und_Tutorial.md)
 - [13 Troubleshooting und FAQ](0_102_10_13_Troubleshooting_und_FAQ.md)
 - [14 Ideen und Roadmap (Future Concepts)](0_102_10_14_Ideen_und_Roadmap.md)
 - [15 Interoperabilität, Export und Multi-Build Formate]
(0_102_10_15_Interoperabilitaet_und_Export.md)
 - [16 Build 12: LightRAG Graph Retrieval Pipeline und Backend-Anbindung]
(0_102_15_build12_lightrag_pipeline_dokumentation.md)
-

01 Philosophie, Kognition und Visuelle Kommunikation

Dieses Kapitel ist das Fundament von Nodges. Es geht nicht um Code oder APIs, sondern um die Frage: **Warum existiert Nodges?** Es bÄ¼ndelt die pÄ¼dagogische Methodik, die psychologischen Konzepte zur visuellen Kommunikation und die DiversitÄ¼t der menschlichen Vorstellungskraft.

1. Die Kernvision: Spatial Analytics statt Flat Data

****Visualisierung:**** Siehe [01_SpatialLiteracyFlow_003.mmd]
(01_SpatialLiteracyFlow_003.mmd)

Nodges ist keine Weiterentwicklung von Excel oder Powerpoint. Herkömmliche Visualisierungen (wie sechs aufeinanderfolgende Balkendiagramme) zwingen den Nutzer zur passiven Suche und linearem Konsum. Das Gehirn wird schnell ermüdet ("Flat Data").

Die Lösung: Spatial Literacy.

Der Mensch ist evolutionär darauf trainiert, räumliche Muster, Distanzen, Ausreißer und Schwärme im dreidimensionalen Raum in Bruchteilen von Sekunden zu erfassen. Nodges übersetzt abstrakte Netzwerke in greifbare 3D-Umgebungen. Der Nutzer blickt nicht mehr *auf* ein Diagramm, sondern bewegt sich explorativ *inmitten* der Daten. Das notorische "Hairball-Problem" (undurchdringbare 2D-Knotenknäuel) wird durch die Tiefendimension (Z-Achse) entzerrt.

2. Executable Storytelling und der Bruch des Monologs

Traditionelle Diagramme sind Monologe des Autors. Nodges etabliert einen interaktiven Diskurs:

- **Scaffolding (Das Lerngerüst):** Der Autor liefert eine kuratierte Basisansicht (ein "Cold Start"-Setup). Er nimmt den Betrachter an die Hand und führt ihn in das System ein. Das verhindert die anfängliche Überforderung vor einem leeren Graphen.
- **Inquiry-based Learning (Forschendes Lernen):** Der Nutzer ist kein passiver Konsument. Er kann die geführte "Show" jederzeit pausieren. Aus Instruktion wird Konstruktion: Der Nutzer ändert das Mapping, filtert Eigenschaften heraus und testet Hypothesen ("Was-wäre-wenn"-Szenarien) live im Raum.

3. Cognitive Offloading (Kognitive Entlastung)

Das Gehirn soll in Nodges keine Text-Panels lesen müssen, um Zusammenhänge zu verstehen.

- **Semantisches Mapping:** Komplexe Sachverhalte (wie politischer Einfluss, Netzwerkstatus, Geldfluss) werden direkt in die visuelle Verarbeitung "ausgelagert".
 - Wenn das System "Einfluss" als *Pulsieren (Breathing Glow)* oder *Größe* darstellt, spürt der Nutzer die Relevanz eines Knotens sofort, ohne einen Zahlenwert parsen zu müssen.
 - **Signal-Rausch-Verhältnis:** Unwichtige Elemente werden nicht hart gelöscht, sondern per `opacity`-Mapping fast unsichtbar (Ghosting-Effekt) gemacht. Der Kontext bleibt erhalten, das Signal tritt kristallklar hervor.
-

4. Die Diversität der Kognition: Textanalyzer vs. Visualizer

Die größte Stärke von Nodges ist die Anpassungsfähigkeit an individuelle Gehirne. Menschen haben fundamental unterschiedliche Herangehensweisen an Probleme.

Textanalyzer

Ein eher logisch-struktureller Denker kann sich in Nodges auf klar definierte Labels, Taxonomien und starre Raster (wie die Achsen-Sperrung) konzentrieren. Er liest die Metadaten.

Visualizer

Ein visuell/assoziativ geprägter Mensch erfasst das System intuitiv über Farben, organische Clusters (Gestaltgesetze der Nähe in der Physik-Engine) und dicke vs. dünne Kanten.

Das Geheimnis der "Inneren Bilder"

Die "inneren Bilder" der Menschen sind hochgradig divers, werden heute aber meist durch klassische PowerPoint-Normen oder Werbeästhetik beschrieben.

- **Die Vision:** Nodges möchte unbeeinflusste, teils unübliche Metaphern zulassen (z.B. Konzepte aus der Synästhesie, Fraktale oder Schwarmverhalten).
 - **Zielgruppe:** Nodges richtet sich damit explizit nicht nur an Systemarchitekten oder Datenanalysten, sondern ist ein Werkzeug für Neurodiversität (z.B. Menschen im Autismus-Spektrum, mit ADHS oder Legasthenie), die in textlastigen Umgebungen benachteiligt sind, im 3D-Sensemaking jedoch brillieren.
-

5. Zielgruppen und Anwendungszwecke

Nodges ist ein Universalwerkzeug zur Kognitionserweiterung:

- **Gedankenstütze:** Eigene mentale Modelle auslagern und räumlich strukturieren.
 - **Kollaboration:** Eine gemeinsame visuelle Sprache als verlässliche Basis für interpersonelle Konfliktlösung oder Design-Entscheidungen finden.
-

6. Die 7 Dimensionen der visuellen Kommunikation

****Visualisierung:**** Siehe [01_SiebenDimensionenVisuell_001.mmd]
 (01_SiebenDimensionenVisuell_001.mmd)

Um das Cognitive Offloading optimal zu nutzen, kann Nodges Informationen theoretisch in **7 gleichzeitigen Dimensionen** codieren. Die "Goldene Regel" lautet jedoch: Maximal 4 Dimensionen gleichzeitig nutzen, um das Gehirn nicht zu ¼berlasten (z.B. Position + Größe + Farbe + Animation).

| Dimension | Visueller Kanal | Wahrnehmung | Eignung |

|---|---|---|---|

| 1. **Raum** | Position (X, Y, Z) | Sofort, preattentiv | 3 kontinuierliche Werte |

| 2. **Größe** | Skalierung (Radius) | Sofort, preattentiv | 1 kontinuierlicher Wert |

| 3. **Farbe** | Hue, Saturation | Sofort, preattentiv | Kategorisch oder Kontinuierlich |

| 4. **Form** | Geometrie | Schnell, bewusst | Kategorien (max. 5-7 Formen) |

| 5. **Bewegung** | Animation (Flow) | Aufmerksamkeitslenkend | Frequenz + Amplitude |

| 6. **Transparenz** | Opacity (Ghosting) | Bewusst, fokussierend | Relevanz, Hintergrundrauschen |

| 7. **Text** | Label | Langsam, lesend | Unbegrenzt, aber analytisch |

7. Die Metaphern-Bibliothek

****Visualisierung:**** Siehe [01_MetaphernBibliothek_002.mmd]
(01_MetaphernBibliothek_002.mmd)

Wie können diese Dimensionen kombiniert werden, um ein System (z.B. eine Firma, ein Netzwerk) für das Gehirn verständlich zu machen? Nodges unterstützt verschiedene Denkmuster (Metaphern):

1. **Die Organismus-Metapher:** (Gut für Ökosysteme/Gesundheit) Knoten sind Organe. Größe = Gesundheit. Farbe = Zustand (grün/rot). Kanten = Nervenbahnen mit Flow-Animationen.

2. **Die Stadt-Metapher:** (Gut für Infrastruktur/Logistik) Knoten = Gebäude (verschiedene Höhen auf der Y-Achse). Position = geografisch (Sperrung der Physik). Kanten = Straßen (Breite entspricht Kapazität).

3. **Die Galaxie-Metapher:** (Gut für Soziale Netzwerke/Einfluss) Knoten = Sterne (leuchtender Glow = Aktivität). Größe = Masse/Einfluss. Kanten = transparente Gravitationsfelder, die Cluster formen.

01 Vision und Systemarchitektur

1. Grundlegende Philosophie und Kognition

Nodges ist nicht einfach eine weitere Dashboard-Anwendung zur Betrachtung tabellarischer Daten. Nodges ist als dreidimensionale, kognitive Explorationsplattform konzipiert. Die grundlegende Vision besteht darin, abstrakte Zusammenhänge, Wissensgraphen und komplexe, multidimensionale Datensätze intuitiv und visuell erfahrbar zu machen.

1.1 Die Überwindung von Flat-Data

Klassische Datenstrukturen (SQL, Excel) zwingen multidimensionale Zusammenhänge in zweidimensionale Tabellen. Nodges bricht mit diesem Paradigma. Durch die Nutzung der räumlichen Dimension (X, Y, Z) gepaart mit visuellen Eigenschaften (Größe, Farbe, Form, Transparenz, Bewegung) ermöglicht Nodges die simultane Wahrnehmung von bis zu sieben Informationsdimensionen.

Der Mensch ist evolutionär darauf trainiert, räumliche Muster, Cluster und Ausreißer im dreidimensionalen Raum in Bruchteilen von Sekunden zu erkennen. Nodges nutzt diese kognitiven Fähigkeiten, um "Sensemaking" in komplexen Systemen zu beschleunigen.

1.2 Die Symbiose aus KI und 3D-Engine

Nodges vereint zwei hochmoderne Technologien zu einem einzigartigen Werkzeug:

- **Large Language Models (LLMs):** Fungieren als "Ontologie-Architekten" und Kuratoren. Sie haben die Aufgabe, Themen semantisch so tief wie möglich zu erschließen, Entitäten zu identifizieren und ihre Beziehungsgeflechte (Taxonomien, Kausalitäten, zeitliche Abläufe) zu extrahieren.
 - **Three.js (WebGL):** Dient als extrem performante Visualisierungsebene, die die vom LLM generierten Graphen in Echtzeit rendert, filtert und physikalisch simuliert.
-

2. Die 7 Dimensionen der Kommunikation

Jedes Element (Knoten oder Kante) in Nodges ist nicht nur ein visuelles Objekt, sondern ein Informationsträger. Die Architektur unterstützt das Mapping von Daten auf folgende sieben Kanäle:

1. **Raum (Position X, Y, Z):** Der stärkste kognitive Kanal. Erlaubt Kategorisierung (z.B. X=Zeit, Y=Relevanz, Z=Geografie).
2. **Größe (Skalierung):** Ideal für kontinuierliche, numerische Metriken (z.B. Umsatz, Einflusswert, Bevölkerung).
3. **Farbe (Hue, Saturation, Lightness):** Hervorragend für diskrete Kategorien (z.B. Status: Rot/Grün, Typ: Firma/Person) oder Heatmaps.
4. **Form (Geometrie):** Kugeln, Würfel, Kapseln zur Unterscheidung von Entitätstypen.
5. **Bewegung (Animation / Flow):** Pulsieren oder fließende Partikel auf Kanten zur Darstellung von Datenfluss, Transaktionen oder zeitlicher Dynamik.
6. **Transparenz (Opacity):** Zur Darstellung von Unsicherheit, Vergangenen oder Herausfiltern irrelevanter Daten.
7. **Text (Label):** Die explizite, textuelle Beschreibung.

[!IMPORTANT]

****Kognitive Belastung (Goldene Regel):**** Es dürfen maximal 4 Dimensionen gleichzeitig intensiv genutzt werden. Werden alle 7 Kanäle gleichzeitig mit verschiedenen Datenpunkten gemappt, entsteht ein kognitiver Overload ("Cluttering"). Die Architektur von Nodges empfiehlt als Best-Practice: Position + Farbe + Größe + 1 Detail.

3. Strikte Trennung von Ontologie, Daten und Visual Mapping (Die Build 5 Revolution)

****Visualisierung:**** Siehe [02_DreiSaeulenArchitektur_001.mmd] (02_DreiSaeulenArchitektur_001.mmd) und [02_Build5PivotEntkopplung_003.mmd] (02_Build5PivotEntkopplung_003.mmd)

Einer der wichtigsten Paradigmenwechsel in der Geschichte von Nodges (eingeführt mit Build 5) ist die strikte Entkopplung der Datenstruktur von der visuellen Repräsentation. Frühere Versionen verknüpften den `type` eines Objekts hart mit seiner Darstellung (z.B. `if (type === 'Server') renderCube()`). Dies limitierte die Flexibilität extrem.

Die moderne Architektur gliedert sich zwingend in drei unabhängige Säulen:

3.1 Die Ontologie (Das Schema)

Definiert die Struktur der Welt. Welche Arten von Knoten (Entities) existieren? Welche Arten von Kanten (Relationships) sind erlaubt? Welche Attribute besitzen sie?

- Die Ontologie ist ein reines Metadaten-Konstrukt.
- Die Eigenschaft `type` dient nur noch der logischen Kategorisierung und Filterung, stellt aber keine visuelle Vorschrift mehr dar.

3.2 Die Daten (Der Graph)

Die tatsächlichen Instanzen.

- Die Datenstruktur ist **vollständig flach**.
- Es gibt keine verschachtelten Objekte innerhalb von Knoten. Besitzt ein Knoten "Kinder", so müssen diese Kinder als eigene Knoten angelegt werden, verbunden durch Kanten.

- Jeder Knoten enthält ein `properties`-Objekt, das maximalen Datenreichtum liefert, völlig unabhängig davon, ob diese Daten initial angezeigt werden oder nicht.

3.3 Das Visual Mapping (Die Repräsentation)

Die dynamische Brücke zwischen Ontologie/Daten und der 3D-Szene.

- Nodges übernimmt die Regie über die Darstellung der gelieferten Rohdaten durch heuristisches Mapping beim Start.
 - Der Nutzer kann das Mapping live in der UI ändern, ohne dass die zugrunde liegenden Rohdaten neu geladen werden müssen.
 - Diese Entkopplung ermöglicht es, exakt denselben Datensatz in völlig unterschiedlichen Kontexten zu betrachten (z.B. denselben Unternehmensgraph einmal geclustert nach Abteilung, und danach geclustert nach Budget-Verantwortung).
-

4. Tech-Stack und Architektur-Übersicht

****Visualisierung:**** Siehe [02_TechStackUebersicht_002.mmd]
(02_TechStackUebersicht_002.mmd)

Das Fundament von Nodges besteht aus einer modernen, leichtgewichtigen Toolchain:

4.1 Frontend Core

- **HTML5 & Vanilla CSS (`index.css`)**: Das Styling nutzt modernes, reines CSS. Auf Frameworks wie Tailwind wird bewusst verzichtet, um die Kontrolle über das Design System (Glassmorphismus, Micro-Animations) maximal zu halten.

- **TypeScript:** Sorgt für strikte Typisierung, die insbesondere bei der Verarbeitung der LLM-Daten durch Interfaces essenziell ist.
- **Vite:** Als extrem schnelles Build-Tool und Dev-Server.

4.2 Engine und Rendering

- **Three.js (^0.161.0):** Die Kern-Engine für das 3D-Rendering. Verwaltet Szene, Kamera, Beleuchtung und das Instanced-Rendering für hohe Performance bei tausenden Knoten.

4.3 Logik und Hilfssysteme

- **Zod (^3.22.4) & zod-to-json-schema:** Zuständig für die Laufzeitvalidierung. Definiert das Schema, zwingt das LLM in ein strukturiertes JSON-Format und validiert die Rückgabe.
- **lil-gui (^0.19.1):** Steuert das Sidebar-Interface und die Mapping-Kontrollen. Es ist extrem performant, DOM-effizient und fügt sich nahtlos in WebGL-Anwendungen ein.
- **Vitest:** Dient der Quality Assurance, Unit-Testing und Coverage-Analyse (`test:ui` , `test:coverage`).

4.4 Architektur-Prinzipien

1. **Zero Data Retention:** Nodges ist eine Frontend-only Applikation. Sie speichert niemals API-Keys im Code oder auf einem Backend-Server. LLM-Provider-Einstellungen werden im LocalStorage des Browsers des Nutzers verwaltet.
2. **Local-First Ready:** Die Architektur unterstützt die nahtlose Umschaltung von Cloud-Inferenz (OpenAI, Anthropic) auf lokale Inferenz (Ollama, LM Studio).
3. **Decoupled UI:** Die Benutzeroberfläche operiert unabhängig von der Render-Schleife (Render-Loop).

02 LLM-Integration und Datenpipelines

Dieses Kapitel widmet sich der komplexesten und mächtigsten Komponente von Nodges: der Datengenerierungs- und Verarbeitungspipeline mittels Large Language Models.

Nodges behandelt LLMs nicht als einfache Chatbots, sondern orchestriert sie als agentische Systeme. Das LLM agiert als "Ontologie-Architekt" und "Daten-Miner", der unstrukturierte Prompts, rohe Texte oder externe Datenquellen analysiert und in hochstrukturierte 3D-Graphen $\frac{1}{4}$ bersetzt.

1. Die Methodik: Der 2-stufige Generierungs-Workflow

****Visualisierung:**** Siehe [03_ZweistufigenGenerierungsWorkflow_001.mmd]
(03_ZweistufigenGenerierungsWorkflow_001.mmd)

Ein zentrales Problem der frühen Nodges-Builds (Build 1-4) waren LLM-Halluzinationen und das Brechen von JSON-Schemata bei komplexen Graphen. Das LLM generierte Attribute in den Daten, die im Schema nie definiert wurden, oder vergaß Relationen.

Die Lösung ist der **2-stufige Workflow**, der ab Build 5 eingeführt und mit Build 10 perfektioniert wurde.

Phase 1: Die Ontologie-Phase (`ontology_prompt.md`)

Bevor auch nur ein einziger Datenpunkt generiert wird, wird das LLM gezwungen, das Regelwerk der neuen Welt zu definieren.

- Der System-Prompt weist das LLM an, das Thema zu analysieren und ein Metadaten-Schema (`dataModel`) sowie visuelle Präferenzen (`visualMappings`) zu entwerfen.
- In dieser Phase bleibt das Array für die eigentlichen `entities` und `relationships` **strikt leer**.
- Das System parst das zurückgegebene JSON und erstellt dynamisch ein Zod-Laufzeitschema aus der vom LLM erfundenen Ontologie.

Phase 2: Die Instanz-Phase

Erst jetzt wird das in Phase 1 entworfene Schema als harter Constraint genutzt.

- Das LLM wird angewiesen, nun die echten Daten (`entities` und `relationships`) zu generieren.
- Durch Zod (und Features wie *Structured Outputs / JSON Schema Mode*) ist das LLM physisch unfähig, Attribute zu erfinden, die es nicht selbst in Phase 1 definiert hat.
- Das Resultat ist ein absolut valider, halluzinationsfreier Datensatz.

2. Die Evolution der Pipelines (Build 6 bis Build 12)

****Visualisierung:**** Siehe 03_PipelineEvolution_003.mmd

Die Architektur von Nodges hat sich evolutionär weiterentwickelt, um unterschiedlichen Anforderungen an Qualität, Geschwindigkeit und Kosten gerecht zu werden.

2.1 Build 6: Der JSON Schema Constraint Mode

Die Basis-Pipeline. Nutzt intensiv Zod, um das LLM auf strukturierte Ausgaben festzunageln. Sehr schnell, sehr zuverlässig für Standardthemen ("Erstelle mir einen Graphen über das Sonnensystem"), greift jedoch nur auf das latente Wissen (Weights) des Modells zu.

2.2 Build 8: Faktencheck und Semantic Web (Wikidata)

Um die "Wahrheit" der Daten zu garantieren und das Modell zu erden (Grounding), wurde Build 8 entwickelt.

- **Ablauf:** Der User gibt einen Prompt ein -> LLM extrahiert Suchbegriffe -> Nodges fragt live über die Wikidata-API SPARQL-Queries ab -> Die harten, verifizierten RDF/SPARQL-Triples kommen zurück -> Das LLM transformiert diese Triples in das Nodges-Schema.
- **Vorteil:** Keine Halluzinationen bei harten Fakten (Geburtsdaten, Distanzen, taxonomische Zugehörigkeiten).

2.3 Build 9: Vektor-Deduplizierung

Bei sehr großen oder iterativ gewachsenen Graphen neigen LLMs dazu, Entitäten doppelt anzulegen (z.B. "USA" und "United States").

- Build 9 nutzt Embeddings (Vektorrepräsentationen von Texten).
- Nach der Generierung vergleicht das System die Vektoren aller Knoten. Knoten mit einer Kosinus-Ähnlichkeit von > 95% werden verschmolzen.

2.4 Build 10: Die modulare, konfigurierbare Pipeline

****Visualisierung:**** Siehe [03_Build10ModulKonfiguration_002.mmd]
(03_Build10ModulKonfiguration_002.mmd)

Build 10 fasst alle bisherigen Erkenntnisse in einem ultimativen, modularen UI zusammen. Der Benutzer ist nicht mehr auf eine starre Pipeline angewiesen, sondern konfiguriert den Datenfluss im `CreatePanel` live.

Die 4 Kern-Module der Build 10 Pipeline:

1. Modell-Quelle (Inferenz-Backend):

- *Cloud*: Routing über OpenRouter, OpenAI, Anthropic für maximale Intelligenz (z.B. GPT-4o, Claude 3.5 Sonnet).
- *Lokal (Local-First)*: Direkte Anbindung an lokale Inferenz-Server wie Ollama (`http://localhost:11434`) oder LM Studio. Garantiert 100% Privacy und null API-Kosten.

2. Grounding-Strategie:

- *Kein Grounding*: Nutzt nur die Modellgewichte (wie Build 6).
- *Wikidata Live*: Faktencheck über SPARQL (wie Build 8).
- *Wikidata Live*: Faktencheck über SPARQL (wie Build 8).
- *RAG (Retrieval-Augmented Generation)*: Erlaubt das Einlesen von URLs oder PDFs als Basis für die Graphgenerierung.
- *Vektor-Deduplizierung*: Nachgelagerte Bereinigung (wie Build 9).

3. Qualitätssicherung (QA & Evaluierung):

- *Keine*: Schneller Single-Pass.

- *Generator + Kritiker (Multi-Pass)*: Ein zweiter LLM-Agent prüft den generierten Graphen auf logische Fehler und korrigiert ihn selbstständig, bevor er an die UI gesendet wird.
- *Human-in-the-loop*: Das Generierungsergebnis wird in einem modalen Fenster abgefangen. Der Mensch reviewt, löscht oder editiert Entitäten, bevor sie in 3D gerendert werden.

4. Kanten-Bewertungsmethode (Edge Weighting):

- *LLM-Schätzung*: Das LLM "rät" die Stärke der Beziehung.
- *Feste Taxonomie*: Kantenstärken sind durch Klassen fest definiert.
- *Kosinus-Ähnlichkeit*: Kantenstärken werden algorithmisch aus der semantischen Nähe der verbundenen Knoten-Embeddings berechnet.

2.5 Build 12: LightRAG Graph Retrieval Pipeline (Lokales Graph-RAG)

Build 12 führt eine vollwertige Graph-RAG Pipeline durch Anbindung an einen Python FastAPI Microservice (`lightrag-backend/main.py`) ein.

- **Dual-Struktur**: Das System verbindet Vektorsuche mit expliziten Graph-Communities.
- **Datenbank-Management**: Unterstützung mehrerer Wissensspeicher und dynamische Umschaltung im FilePanelUI.
- **Relation Set Normalisierung**: Automatische Bereinigung und Zuordnung unstrukturierter Beziehungs-Kategorien im DataParser auf das Nodges Schema 5.2.

3. Architektur der LLM-Integration im Code

3.1 `LLMService.ts` als Single Source of Truth

Alle Kommunikation mit der Außenwelt (APIs, lokale LLMs, Vektor-Stores) läuft zwingend über den `LLMService`.

- **Verbot redundanter Fetches:** Es dürfen keine `fetch`-Calls an Provider irgendwo in UI-Komponenten oder im SceneManager platziert werden.
- **Provider-Abstraktion:** Das restliche System muss nicht wissen, ob Claude 3.5 oder Llama-3 lokal antwortet. `LLMService` normalisiert Inputs und Outputs.

3.2 Error Handling und Resilience

****Visualisierung:**** Siehe [03_LLMServiceFehlerbehandlung_004.mmd]
(03_LLMServiceFehlerbehandlung_004.mmd)

LLMs sind stochastische Systeme; Netzwerkfehler oder Ratelimits (HTTP 429) sind alltäglich.

- `LLMService` implementiert Exponential Backoff und Retry-Logiken.
- Fällt ein Cloud-Provider aus, kann das System heuristisch vorschlagen, auf einen konfigurierten lokalen Provider (Ollama) als Fallback umzuschalten.
- Bei Fehlern (z.B. Zod Schema-Validation schlägt fehl) stoppt das Frontend nicht ab (White-Screen of Death). Der Fehler wird über den zentralen `ErrorHandler` an die UI (Toasts/Modals) delegiert, und die alte 3D-Szene bleibt erhalten (`clearScene` wird nur bei Erfolg getriggert).

03 Datenmodell und Validierung

Dieses Kapitel beschreibt tiefgreifend die Datenstruktur, auf der Nodges aufbaut, und wie die Integrität dieser Daten beim Import, Export oder bei der Generierung durch LLMs sichergestellt wird.

1. Das Graphen-Paradigma: Flach und Beziehungsorientiert

****Visualisierung:**** Siehe [04_FlatvsDeeplyNested_003.mmd]
(04_FlatvsDeeplyNested_003.mmd)

Ein Kernparadigma von Nodges ist die strikte, flache Organisation der Graphendaten. Viele traditionelle JSON-Strukturen oder NoSQL-Datenbanken neigen zu tief verschachtelten Dokumenten. In einer 3D-Graphumgebung ist dies ein Anti-Pattern.

1.1 Verbot von Verschachtelungen (Nesting)

- Zugehörigkeiten, hierarchische Abhängigkeiten und semantische Zusammenhänge dürfen **niemals** als verschachtelte Objekte oder Listen innerhalb eines Knotens gespeichert werden.

- *Falsch:* Ein Knoten "Firma" enthält ein Array `employees: [{id: 1, name: "Alice"}, {id: 2, name: "Bob"}]`.

- *Korrekt:* Es existiert ein Knoten "Firma". Es existieren Knoten "Alice" und "Bob". Es existieren Kanten "works_for" von Alice zur Firma und von Bob zur Firma.

- **Warum?** Eine Physik-Engine oder ein 3D-Layout kann verschachtelte Arrays nicht räumlich anordnen. Nur echte, flache Kanten erlauben das Spannen physikalischer Federn (Forces) zwischen Knoten und ermöglichen visuelles Clustering.

1.2 Beziehungen als eigene Entitäten (First-Class Edges)

Jede Beziehung zwischen zwei Datenpunkten ist zwingend eine eigene Kante (`Edge`). Dies ermöglicht es, der Kante selbst Eigenschaften zuzuweisen (z.B. `weight` , `date_established` , `type`), die wiederum visualisiert werden können (Dicke, Farbe, gestrichelte Linie).

2. Das JSON-Datenformat (Schema 5.0 / Build 5+)

****Visualisierung:**** Siehe [04_JsonSchemaStruktur_001.mmd]
(04_JsonSchemaStruktur_001.mmd)

Alle von den Pipelines erzeugten und von Nodges geladenen Daten müssen einem strikten JSON-Format entsprechen. Dieses Format ist in vier logische Blöcke unterteilt:

```
{
  "dataModel": {
    "nodeTypes": [
      { "id": "person", "label": "Person", "properties": [...] }
    ],
    "edgeTypes": [
      { "id": "knows", "label": "Kennt", "properties": [...] }
    ]
  },
  "visualMappings": {
    "nodes": {
```

```

    "color": { "property": "type", "scale": "categorical" },
    "size": { "property": "influence", "scale": "linear" }
  },
  "edges": {}
},
"entities": [
  {
    "id": "node_1",
    "type": "person",
    "label": "Alice",
    "properties": { "influence": 80, "age": 30 }
  }
],
"relationships": [
  {
    "id": "edge_1",
    "source": "node_1",
    "target": "node_2",
    "type": "knows",
    "properties": { "weight": 5 }
  }
]
}

```

Die Komponenten:

1. **dataModel (Ontologie):** Definiert die vom LLM (oder Benutzer) erdachten Klassen für Knoten und Kanten sowie deren erlaubte Properties.
2. **visualMappings (Präferenz):** Vorschriften, wie bestimmte Attribute initial gerendert werden sollen (z.B. "Nutze `influence` für die Größe").
3. **entities (Knoten):** Das flache Array der Datenpunkte.
4. **relationships (Kanten):** Das flache Array der Verbindungen. Referenziert `entities` zwingend über deren `id` in den Feldern `source` und `target`.

2.1 Das Temporale Objekt (Zeitdimension)

****Visualisierung:**** Siehe [04_TemporalesObjektKeyframes_004.mmd]
(04_TemporalesObjektKeyframes_004.mmd)

Jeder Knoten (`Entity`) und jede Kante (`Relationship`) besitzt ein optionales `temporal` Objekt. Dies ist die Grundlage für die Visualisierung von Graphen über die Zeit (TimeEngine).

```
"temporal": {
  "validFrom": 1990,
  "validTo": 2025,
  "history": [
    {
      "timestamp": 2000,
      "changes": { "influence": 95 }
    }
  ]
}
```

- `validFrom` / `validTo` : Definiert die pure Existenz des Elements im Zeitstrahl (Wann wird es gerendert, wann stirbt es).
- `history` : Ein Array von Keyframes. Statt Daten redundant zu duplizieren, speichert das `changes` -Objekt nur **Deltas** (Werte, die sich genau zu diesem Timestamp ändern).

3. Runtime-Validierung mittels Zod

Da Nodges Daten oft dynamisch über APIs von stochastischen Modellen (LLMs) bezieht, ist Laufzeitvalidierung (Runtime Validation) kritisch. TypeScript Interfaces prüfen nur zur Compile-Zeit, sie schützen nicht vor defekten JSON-Payloads zur Laufzeit.

3.1 Zod-Schemas als T rsteher

- Die gesamte Datenstruktur (`GraphData` , `Node` , `Edge`) wird im Code durch strikte **Zod-Schemas** (`z.object({...})`) definiert.
- Jedes JSON, das in das System geladen wird (per Datei-Upload, Drag & Drop, oder LLM-Response), wird durch `GraphDataSchema.parse()` gepresst.
- Fehlen Felder (z.B. eine Kante referenziert eine `source` -ID, die nicht existiert), wirft Zod einen synchronen Fehler, bevor die Daten die 3D-Engine erreichen k nnen. Dies verhindert undefiniertes Verhalten oder White-Screens (Speicherzugriffsfehler in Three.js).

3.2 Structured Outputs (JSON Schema Mode)

Nodges  bersetzt sein internes Zod-Schema  ber Tools wie `zod-to-json-schema` in ein standardisiertes JSON-Schema. Dieses wird bei API-Aufrufen (z.B. an OpenAI) im Parameter `response_format` oder als System-Prompt mitgeliefert. Das zwingt das Modell deterministisch in die geforderte Struktur.

4. Der Lebenszyklus der Daten (`App.loadGraphData`)

****Visualisierung:**** Siehe [04_LoadGraphDataLebenszyklus_002.mmd]
(04_LoadGraphDataLebenszyklus_002.mmd)

Der Datenfluss im Client ist streng gerichtet. `App.loadGraphData` ist der einzige Einstiegspunkt (Flaschenhals) f r neue Netzwerke.

Die Kette der Ereignisse:

1. **Ingestion & Parse:** Roher String wird via `JSON.parse` gelesen.
 2. **Validation:** Zod prüft die Struktur.
 3. **State Update:** Der `GraphDataManager` (die interne "Datenbank") wird bereinigt und mit den neuen Entitäten und Beziehungen bestückt.
 4. **Scene Cleardown:** Der `SceneManager` wird angewiesen, die alte 3D-Szene sicher zu leeren (`clearScene`). Dabei müssen alle Geometrien, Materialien und Texturen via `.dispose()` aus dem GPU-Speicher (VRAM) gelöscht werden, um Memory-Leaks zu verhindern.
 5. **Re-Instantiation:** Der `SceneManager` baut die neuen 3D-Meshes auf.
 6. **UI Sync:** Das `MappingPanel` wird über die neue Ontologie (`dataModel`) informiert und baut die Dropdown-Menüs (Slider, Color-Picker) basierend auf den verfügbaren Properties dynamisch neu auf.
-

5. Export und Interoperabilität (GEXF)

Nodges ist kein Daten-Silo. Historisch (siehe ADRs) wurde evaluiert, wie Graphen mit wissenschaftlichen Tools (Gephi, Cytoscape) ausgetauscht werden können.

Während das interne Format JSON ist, zielt die Architektur darauf ab, Interoperabilität mit dem XML-basierten GEXF-Format (Graph Exchange XML Format) zu wahren, um den Import von akademischen Datensätzen oder den Export für tiefere Netzwerkkalkulationen (Betweenness Centrality, PageRank) in Desktop-Tools zu ermöglichen.

04 3D-Rendering und Szenen-Management

Die grafische Darstellung ist das Herzstück von Nodges. Dieses Kapitel taucht tief in die Implementierung der 3D-Engine mittels **Three.js** ein und erläutert die Strategien (Instancing, TubeGeometry), mit denen wir zehntausende Objekte flüssig im Browser rendern.

1. Three.js Integration & Setup

****Visualisierung:**** Siehe [05_ThreeJsSzenenGraph_001.mmd]
(05_ThreeJsSzenenGraph_001.mmd)

Nodges kapselt den WebGL-Renderer vollständig im `SceneManager`.

1.1 Der Szenen-Graph

- **Scene:** Die Wurzel aller Objekte.
- **Camera:** Eine `PerspectiveCamera` (FOV 75°). Die `Far`-Clipping-Plane wird dynamisch an die Bounding-Box des Graphen angepasst, um auch riesige Netzwerke vollständig darzustellen, ohne Z-Fighting Fehler bei nahen Objekten zu riskieren.
- **Renderer:** Der `WebGLRenderer` ist hochoptimiert:
- `antialias: true`: MSAA-Glättung von Kanten.
- `logarithmicDepthBuffer: true`: Essentiell für Netzwerke mit extremen Größenunterschieden, um Flackern (Z-Fighting) bei überlappenden Geometrien zu

verhindern.

1.2 Beleuchtung (Lighting Strategy)

Licht ist Information. Wir nutzen ein 3-Punkt-Setup für plastische Wahrnehmung:

1. **AmbientLight**: Weiches Grundlicht zur Schattenaufhellung (Intensität 0.4).
 2. **DirectionalLight (Key Light)**: Simuliert hartes gerichtetes Licht, wirft Schatten und definiert die Form (Specular Highlights) auf den Knoten.
 3. **HemisphereLight**: Simuliert Himmels- und Boden-Reflektion für einen natürlicheren Look der Materialien.
-

2. High-Performance Rendering: Instancing

****Visualisierung:**** Siehe [05_InstanceMeshDataflow_002.mmd]
(05_InstanceMeshDataflow_002.mmd)

Ein Graph mit 10.000 Knoten würde den Browser bei der Erstellung von 10.000 separaten Mesh-Objekten in die Knie zwingen (10.000 Draw Calls an die GPU).

2.1 Die Lösung: `InstancedMesh`

Nodges nutzt aggressiv **Instancing**.

- **Konzept**: Die Geometrie (z.B. eine Kugel) und das Material werden nur *ein einziges Mal* an die Grafikkarte gesendet.

- **Instanzen:** Die CPU sendet lediglich ein großes Array (Buffer) mit Transformationsmatrizen (Position, Rotation, Skalierung) und Farben.
- **Resultat:** Die GPU zeichnet alle 10.000 Knoten in einem einzigen Draw Call.

2.2 Implementation (`NodeManager`)

1. **Dummy Object:** Ein temporäres unsichtbares `Object3D` ("Dummy") berechnet die Positionen und Skalierungen.
 2. **Matrix Updates:** `dummy.updateMatrix()` generiert die 4x4 Matrix.
 3. **Buffer Write:** Diese Matrix wird in den GPU-Buffer des `InstancedMesh` an Index `i` geschrieben (`setMatrixAt(i, matrix)`).
 4. **Trigger:** `instanceMatrix.needsUpdate = true` signalisiert Three.js, das Array im nächsten Render-Frame zur GPU zu pushen.
-

3. Kanten-Visualisierung: TubeGeometries

Kanten (Edges) sind deutlich komplexer als Knoten, da sie zwei Koordinaten im Raum verbinden und verformbar sein müssen.

- **Kurven-Logik:** Anstelle gerader Linien berechnet Nodges **Bezier-Kurven** (`QuadraticBezierCurve3`) zwischen Start- und Zielknoten.
- **Multi-Edge Support:** Sind zwei Knoten durch *mehrere* Kanten verbunden (z.B. "kauft bei" und "arbeitet für"), bekommt jede Kante einen algorithmischen Offset im Kontrollpunkt der Kurve. Sie legen sich "bauchig" nebeneinander, wie Kabelstränge, statt sich zu überschneiden.

- **TubeGeometry**: F  r Kanten mit hoher Bedeutung (`weight`) wird statt einer d  nnen Linie ein 3D-Zylinder (`TubeGeometry`) entlang der B  zier-Kurve extrudiert, was visuelle Plastizit  t schafft.

4. Render-Loop Optimierung

****Visualisierung:**** Siehe [05_RenderLoopOptimierung_003.mmd]
(05_RenderLoopOptimierung_003.mmd)

Die `requestAnimationFrame`-Schleife (`animate()`) muss in $< 16\text{ms}$ abschlie  en, um 60 FPS zu garantieren.

- **Keine Garbage Collection**: Im Loop werden niemals Objekte mit `new` erstellt (Vermeidung von GC-Rucklern). Statische Hilfsvariablen (z.B. `_tempVector = new THREE.Vector3()`) werden global recycelt.
- **Frustum Culling**: Three.js pr  ft, ob Objekte im Sichtfeld der Kamera sind (anhand von Bounding-Spheres).
- **On-Demand Rendering**: Wenn sich die Kamera nicht bewegt und die Physik-Simulation zur Ruhe gekommen ist, stoppt Nodges den GPU-Loop, um Laptop-Batterien zu schonen.

5. Das Entkoppelte Visual Mapping (Ab Build 5)

Die Engine besitzt keine hartkodierte Regeln f  r Datentypen mehr (keine `if (type === 'Server')` Logik).

- Alle Knoten starten als `global_node` (Generisches Sphere-Mesh im InstancedMesh).
 - Das System parst die `visualMappings` (aus JSON oder UI) und ¼berschreibt die Scale- und Color-Werte im Matrix-Buffer dynamisch basierend auf den Rohdaten (`properties`) der EntitÄt.
-

05 BenutzeroberflÄche und Interaktion

Das Interface-Design von Nodges zeichnet sich durch die Philosophie der "Progressiven Offenlegung" (Progressive Disclosure) aus. Komplexe Datengraphen erfordern ein UI, das die kognitive Last des Nutzers minimiert. Dieses Kapitel beschreibt den technischen Aufbau und die UX-Konzepte der BenutzeroberflÄche.

1. Design System und Ästhetik

****Visualisierung:**** Siehe [06_UIKomponentenStruktur_001.mmd]
(06_UIKomponentenStruktur_001.mmd)

1.1 Glassmorphismus und Premium-Feel

Nodges verwendet eine moderne, transparente Designsprache.

- **Glassmorphism:** UI-Panels (Sidebar, Modals) wirken transparent, durchbrochen von subtilem `backdrop-filter: blur(10px)`. Dies stellt sicher, dass die 3D-Szene im Hintergrund stets sichtbar bleibt und das Interface nicht isoliert vom Datenraum wirkt.
- **Vanilla CSS (`index.css`):** Um absolute Kontrolle über das Rendering und die Transitions zu behalten, verzichtet Nodges auf Utility-Frameworks wie TailwindCSS. Alle Layouts werden über natives CSS-Grid und Flexbox gesteuert.
- **Micro-Animations:** Hover-Zustände, Klicks auf Buttons oder das Aufklappen von Dropdowns werden von 200ms bis 300ms Transitions (z.B. ease-out) begleitet. Dies verleiht der Applikation ein "lebendiges" Premium-Gefühl.

1.2 Die lil-gui Integration

Für die technische Umsetzung des Sidebars setzt Nodges massiv auf die Bibliothek `lil-gui` (den spirituellen Nachfolger von `dat.gui`).

- `lil-gui` ist extrem performant, DOM-effizient und der De-facto-Standard in der Three.js-Community für Parameter-Tuning.
- Es ermöglicht das schnelle Prototyping und dynamische Rendern von Slidern, Farbpipetten und Dropdowns, direkt gekoppelt an den internen State des `GraphDataManager`.

2. Die Decoupled Mapping UI

****Visualisierung:**** Siehe [06_DecoupledMappingUI_003.mmd]
(06_DecoupledMappingUI_003.mmd)

In traditionellen Daten-Tools muss der User oft kryptische Pfade oder feste Bezeichner in Textfelder tippen, um Daten an visuelle Eigenschaften zu koppeln. Nodges dreht diesen Spieß um.

2.1 Dynamische Ontologie-Vorschläge

Das Mapping-UI in Nodges ist *decoupled* – es kennt keine harten Variablen.

Wenn ein neuer Graph geladen wird, analysiert ein Background-Skript die `properties` aller Knoten.

Die UI generiert daraufhin Dropdown-Menüs (z.B. für die Farbe). Anstatt ein leeres Feld anzubieten, zeigt das Dropdown exakt die Felder an, die im Datensatz existieren (z.B. "Jahresumsatz", "Kategorie", "Region").

2.2 Positionierung: Ein First-Class Channel

Eine Besonderheit der UI ist die Behandlung räumlicher Koordinaten.

- Früher gab es für X, Y und Z jeweils eine eigene, verwirrende Dropdown-Reihe.
- Heute bietet die UI eine gebänderte 'Position'-Eigenschaftsbox an. Wählt der Nutzer ein Datenfeld aus (z.B. "Gründungsjahr"), erscheint dynamisch ein Toggle, mit dem er bestimmen kann, auf welche Achse (oder Achsenkombination) dieser Wert gemappt werden soll. Das räumt die UI drastisch auf.

3. Das CreatePanel (Pipeline-Steuerung)

Das `CreatePanel` ist das Herzstück der Datengenerierung. Hier orchestriert der Nutzer das LLM.

3.1 Build 10 Modul-Konfigurator

Wählt der Nutzer die "Build 10" Pipeline aus, entfaltet sich der dynamische `build10ConfigContainer`. Dieser Container ist das Kontrollzentrum der agentischen Workflow-Konfiguration.

Der Nutzer konfiguriert:

- **Provider-Toggle:** Ein Switch zwischen "Cloud" (OpenRouter) und "Local" (Ollama). Wird "Local" gewählt, blendet die UI automatisch das API-Key-Feld aus, da es nicht benötigt wird.
- **Grounding-Select:** Auswahl zwischen Wikidata-Live-Abfrage, Deduplizierung oder purem LLM-Wissen.
- **Evaluations-Methode:** Aktivieren der Kosinus-Ähnlichkeit für semantische Kantenmarken.

3.2 Human-in-the-loop (Das Review Modal)

Ein kritischer Interaktions-Workflow ist die Qualitätssicherung.

****Visualisierung:**** Siehe [06_HumanInTheLoopWorkflow_002.mmd]
(06_HumanInTheLoopWorkflow_002.mmd)

Ist das "Human-in-the-loop"-Modul aktiv, darf das System nach der LLM-Generierung nicht direkt in die 3D-Szene rendern.

1. Die App friert den Render-Prozess ein.
2. Ein HTML-Modal überlagert die Szene (Glassmorphism-Overlay).
3. Zwei tabellarische Listen werden gerendert: Entitäten und Relationen.
4. Der User fungiert als "Kritiker". Er kann direkt in der Tabelle fehlerhafte Halluzinationen löschen oder neue Entitäten hinzufügen.
5. Erst der Klick auf "Visualisieren" gibt den State an den `GraphDataManager` frei.

4. Feedback-Systeme (Toasts & Loading)

Da LLM-Anfragen teilweise 10 bis 40 Sekunden dauern können (besonders bei lokalen Modellen oder komplexen Wikidata-SPARQL-Ketten), ist Nutzer-Feedback unerlässlich.

- Nodges nutzt ein asynchrones Toast-Notification-System.
- Warnungen (Ratelimits) sind gelb/orange, kritische Zod-Validierungsfehler rot, Erfolge grün.
- Eine Spinner-Overlay-Komponente mit Statusmeldungen (z.B. "Generiere Ontologie...", "Frage Wikidata ab...", "Generiere Embeddings...") hält den Nutzer über den exakten Schritt der Pipeline informiert.

06 Algorithmen, Layout Engine und Physik

Die räumliche Anordnung (Layout) von Graphen ist eines der komplexesten Themen der Informationsvisualisierung. Dieses Kapitel detailliert, wie Nodges klassische Physik-Simulationen mit direktem Daten-Mapping und semantischer Algorithmik kombiniert.

1. Die Rolle der Physik-Engine

****Visualisierung:**** Siehe [07_PhysikMappingEntflechtung_001.mmd]
(07_PhysikMappingEntflechtung_001.mmd)

In klassischen Graph-Viewern (wie Gephi oder alten D3.js Demos) bestimmt ausschließlich eine physikalische Simulation (Force-Directed Graph), wo ein Knoten im Raum liegt. Nodges bricht mit diesem Paradigma, da Position (`x` , `y` , `z`) hier ein First-Class Channel für Daten ist.

1.1 Physik als intelligentes Fallback

Die Physik-Engine (oft implementiert über Repulsion/Attraction-Kräfte, ähnlich dem Barnes-Hut-Algorithmus oder D3-Force 3D) wird **nur dann und genau dort** angewendet, wo der Nutzer keine expliziten Daten gemappt hat.

- **Szenario A (Vollständiges Mapping):** Der Nutzer mappt "Jahresumsatz" auf Y und "Gründungsjahr" auf X. Die Physik-Engine wird für diese Knoten auf der X- und Y-Achse hart deaktiviert (die Koordinaten sind gesperrt).
- **Szenario B (Teilweises Mapping):** Z bleibt ungemappt. Die Physik-Engine darf die Knoten nur noch auf der Z-Achse verschieben (1D-Force-Layout), um Überlappungen auf der Z-Ebene zu vermeiden.
- **Szenario C (Kein Mapping):** Das System operiert im klassischen Modus. Knoten stoßen sich gegenseitig ab (Charge), während Kanten sie wie Federn zusammenziehen (Link Force). Das Ergebnis ist ein organisches, topologisches Cluster.

Dieses Entflechten von Mapping und Layout-Simulation verhindert das chaotische "Herumfliegen" von Knoten, wenn feste Koordinaten gefordert sind.

2. Dimensionalitätsreduktion und semantische Algorithmen

****Visualisierung:**** Siehe [07_KosinusAehnlichkeitPipeline_002.mmd]
(07_KosinusAehnlichkeitPipeline_002.mmd)

Mit dem Sprung zu Build 9 und Build 10 wurde das Algorithmik-Portfolio von Nodges um maschinelles Lernen (NLP/Embeddings) erweitert, um nicht nur quantitative, sondern semantische Zusammenhänge berechnen zu können.

2.1 Vektor-Embeddings

Nodges kann Texte (z.B. die `description` oder das `label` eines Knotens) durch ein Embedding-Modell (z.B. `text-embedding-3-small` von OpenAI oder via Ollama lokal) in hochdimensionale Vektoren übersetzen. Ein Vektor ist eine Liste von Fließkommazahlen (oft 1536 Dimensionen), die die exakte semantische Bedeutung des Wortes repräsentiert.

2.2 Kosinus-Ähnlichkeit (Cosine Similarity)

Sobald Vektoren vorliegen, nutzt Nodges die Kosinus-Ähnlichkeit, um den Winkel zwischen zwei Vektoren zu berechnen.

Die Formel: $\text{similarity} = \cos(\hat{A}, \hat{B}) = (A \cdot B) / (|A| |B|)$

Dies liefert einen Wert zwischen -1 (komplett gegensätzlich) und 1 (identisch).

Anwendung in Nodges (Build 10 Edge Weighting):

- Das System berechnet die Kosinus-Ähnlichkeit zwischen zwei verbundenen Entitäten.

- Der Wert wird normalisiert (z.B. auf eine Skala von 0-100).
- Dieser errechnete Score wird direkt als Eigenschaft (z.B. `semantic_weight`) auf die Kante geschrieben.
- Das Visual Mapping greift diesen Score auf und zeichnet hoch-Ähnliche Kanten dicker, dunkler oder leuchtender. So offenbart die Layout-Engine verborgene semantische Verwandtschaften, die das LLM in der bloßen Taxonomie vielleicht übersehen hätte.

2.3 Deduplizierung (Build 9 Ansatz)

Derselbe Algorithmus wird genutzt, um LLM-Halluzinationen (Duplikate) zu bereinigen. Iteriert die Engine über den Graphen und findet zwei Knoten (z.B. "USA" und "United States of America") mit einer Kosinus-Ähnlichkeit > 0.95 , fusioniert (mergt) der Algorithmus die beiden Knoten und leitet alle eingehenden und ausgehenden Kanten auf den verbleibenden Knoten um.

3. Zukünftige algorithmische Potenziale

****Visualisierung:**** Siehe [07_NetzwerkMetrikenPotenzial_003.mmd]
(07_NetzwerkMetrikenPotenzial_003.mmd)

Die Architektur von Nodges ist darauf vorbereitet, klassische Netzwerk-Metriken lokal im Browser zu berechnen (z.B. via WebWorkers, um den Main-Thread nicht zu blockieren):

- **PageRank:** Identifikation der wichtigsten Knoten basierend auf der Kanten-Topologie.
- **Betweenness Centrality:** Finden von Knoten, die als "Brücken" zwischen verschiedenen Clustern fungieren.

- Diese errechneten Metriken können wiederum als neue `properties` in den Datenbestand injiziert und visuell gemappt werden.

07 Visuelle Effekte und Feedback-Systeme

Ein intuitives visuelles Feedback ist entscheidend, um dem Benutzer Orientierung in der komplexen 3D-Welt zu geben. Nodges implementiert hierfür ein mehrschichtiges System aus Highlights, Animationen und atmosphärischen Effekten.

1. Das Highlight-System (`HighlightManager`)

Der `HighlightManager` ist die zentrale Instanz für alle visuellen Hervorhebungen. Er stellt sicher, dass sich verschiedene Interaktionszustände (z.B. "Ich fahre über einen Knoten" vs. "Ich habe diesen Knoten markiert") nicht gegenseitig visuell zerstören.

1.1 Zustandssicherung und Backup

Bevor ein Objekt visuell verändert wird, legt der Manager ein Backup des originalen Materials an. Beim Entfernen des Highlights wird dieser Originalzustand exakt wiederhergestellt. Das verhindert "Geister-Materialien" (Material-Leaks).

1.2 Prioritäts-Kaskade

Da ein Objekt mehrere Gründe haben kann, hervorgehoben zu werden, existiert eine strikte Kaskade:

1. **SELECTION** (Höchste Prio): Der Benutzer fokussiert dieses Objekt aktiv.
 2. **SEARCH**: Das Objekt ist Teil eines Suchergebnisses (via UI).
 3. **PATH**: Das Objekt liegt auf einem berechneten Shortest-Path.
 4. **HOVER** (Niedrigste Prio): Die Maus streift das Objekt kurzzeitig.
-

2. Highlight-Modi im Detail

****Visualisierung:**** Siehe [08_HighlightPrioritätsKaskade_001.mmd]
(08_HighlightPrioritätsKaskade_001.mmd)

2.1 HOVER-Modus (Feedback der Erreichbarkeit)

- **Zweck:** Signalisiert: "Dieses Objekt ist anfassbar".
- **Visuell (Nodes):** Helligkeitsboost (+30%) der Grundfarbe.
- **Visuell (Edges):** Die Kante wird visuell hervorgehoben und leuchtet bläulich.

2.2 SELECTION-Modus (Der Fokus)

- **Zweck:** Dauerhafte Markierung für Detailanalyse (Öffnet das Sidepanel).
- **Visuell:** Ein kräftiger Glow-Effekt über die `emissive`-Color Eigenschaft des Three.js-Materials.

- **Besonderheit:** Selektierte Objekte "atmen" (Pulsation), um sie vom statischen Rest des Graphen abzuheben.

2.3 SEARCH & PATH (Semantische Highlights)

- **Search:** Nutzt starke Kontrastfarben (z.B. Gelb/Magenta), um Treffer im "Dschungel" tausender Knoten sofort aufblinken zu lassen.
- **Path:** Markiert Ketten von Kanten. Für maximale Lesbarkeit nutzt das System hier den **Ghosting-Effekt:** Alle nicht-beteiligten Knoten und Kanten des Graphen erhalten ein transparentes Material (`opacity: 0.1`), wodurch der Pfad optisch stark in den Vordergrund rückt.

3. Shader und Atmosphärische Effekte

****Visualisierung:**** Siehe 08_ShaderUndEffekte_002.mmd

3.1 Der "Breathing" Glow-Effekt

Das Pulsieren selektierter Knoten wird mathematisch über einen **Sinus-Oszillator** direkt im Render-Loop (oder per Custom Shader) gesteuert:

```
Intensity = Base + Amplitude sin(Time Frequency)
```

- Standardmäßig atmen Objekte mit 0.5 Hz. Dieser fließende Übergang wirkt beruhigend und hochwertig (Premium Aesthetics) im Vergleich zu hartem Blinken.

3.2 Halo-Technik (Outline Rendering)

Echte Outlines sind in WebGL performancelastig, da sie komplexe Post-Processing PÄsse (z.B. Sobel Filter) benÄtigen. Nodges nutzt oft eine performante geometrische Alternative:

- Es wird ein zweites Mesh (Klon) erzeugt, das ca. 40% grÄÄer ist.
 - Dieses erhÄlt `opacity: 0.3` und `depthWrite: false`.
 - Dadurch wird der "Halo" immer hinter dem eigentlichen Objekt gerendert und erzeugt einen weichen, geisterhaften Schein, der kaum GPU-Rechenzeit kostet.
-

08 Interaktions-Design und Input-Processing

Eine nahtlose Interaktion ist fÄr die User Experience in einer 3D-Welt entscheidend. Dieses Kapitel beschreibt, wie Nodges abstrakte Benutzereingaben (Maus, Touch, Keyboard) performant in prÄzise Aktionen innerhalb der visualisierten Daten Äbersetzt.

1. Raycasting: Die BrÄcke zwischen 2D und 3D

****Visualisierung:**** Siehe [09_InputSystemUebersicht_002.mmd]
(09_InputSystemUebersicht_002.mmd)

Das Hauptproblem der 3D-Interaktion auf Bildschirmen: Der Monitor ist flach (2D), die Daten sind räumlich (3D). Nodges nutzt den **Raycasting-Algorithmus** von Three.js (`THREE.Raycaster`), um festzustellen, wohin der User "in die Tiefe" klickt.

1.1 Der mathematische Prozess

1. **Normalisierung:** Die X/Y-Position der Maus (in Pixeln) wird in den Normalized Device Coordinate (NDC) Raum (-1 bis +1) umgerechnet.
2. **Strahl-Projektion:** Der Raycaster schießt einen unsichtbaren Vektor von der Kameraposition durch den transformierten "Mauspunkt" auf der Linse tief in die Unendlichkeit der 3D-Szene.
3. **Intersektion:** Die Engine berechnet Schnittpunkte des Strahls mit den Bounding-Boxen und Flächen aller Objekte.
4. **Sortierung:** Die Liste der getroffenen Objekte wird nach ihrer Entfernung zur Kamera sortiert; das vorderste sichtbare Objekt gewinnt.

1.2 Performance-Herausforderung bei InstancedMesh

Da ein `InstancedMesh` (z.B. für 10.000 generische Knoten) für Three.js technisch nur *ein* einziges Objekt ist, liefert ein normaler Raycast als Ergebnis nur: "Du hast das große Array getroffen".

Nodges fragt daher die Eigenschaft `instanceId` des Raycast-Intersects ab, um den exakten Matrix-Index der getroffenen Kugel zu extrahieren. Dieser Index wird über eine interne Lookup-Table blitzschnell auf die ursprüngliche `id` des Datenknotens zurückgemappt.

2. Der `CentralEventManager` (CEM)

****Visualisierung:**** Siehe [09_RaycastingUndCEM_001.mmd]
(09_RaycastingUndCEM_001.mmd)

Der CEM ist das "Gehirn" der Interaktion. Er sorgt für eine saubere Trennung zwischen Hardware-Events (`mousemove` , `pointerdown`) und der Business-Logik (`selectNode` , `hoverEdge`).

2.1 Intelligente Click-Erkennung ("Click vs. Drag")

Ein Standardproblem in 3D-Viewern: Wenn ein User die Kamera dreht (Drag-Bewegung), lässt er die Maus oft unbeabsichtigt über einem Knoten los. Ein naiver Event-Handler (`mouseup` oder `click`) würde dies fälschlicherweise als "Selektion des Knotens" interpretieren.

Die Lösung: Nodges puffert die Mauskoordinaten bei `mousedown` . Bei `mouseup` wird die Vektor-Distanz berechnet.

- Distanz < 5 Pixel? -> **Klick** (Raycast für Selektion auflösen).
- Distanz > 5 Pixel? -> **Drag** (Es war nur eine Kamera-Drehung, Klick wird ignoriert).

2.2 Hover-Throttling

Raycasting bei *jedem einzelnen Pixel*, den die Maus bewegt, würde den CPU-Thread lahmlegen.

- Der CEM drosselt (throttles) das `mousemove` -Event auf ca. 50ms (20 Checks pro Sekunde).
- Das ist für das menschliche Auge flüssig genug, spart aber massive GPU/CPU-Zyklen für den Render-Loop.

3. Kamera-Steuerung und "Cinematic UI"

Die Kamera (`THREE.PerspectiveCamera`) wird nativ über `OrbitControls` gesteuert (Orbit, Pan, Zoom). Wir haben sie jedoch mit kinoreifen UX-Verbesserungen angereichert.

3.1 Auto-Focus & Fly-To Animationen

Wenn ein Benutzer im UI ein Suchergebnis auswählt, springt die Kamera nicht hart auf die neuen Koordinaten.

- **TWEEN.js Integration:** Nodges berechnet den Zielvektor und interpoliert die Kameraposition sanft dorthin.
- **Easing:** Die Bewegung nutzt "Smooth-Step" Funktionen (langsam starten, schnell in der Mitte, sanft abbremsen) für ein kinoreifes Gefühl.
- **Abstandshaltung:** Die Kamera hält an der Bounding-Sphere des Objekts an, statt im Kern der Geometrie zu enden.

3.2 Kontext-Sensitiver Zoom

Beim Zoomen mit dem Mausrad berechnet Nodges den Zoom-Faktor basierend auf der aktuellen Entfernung zum Ziel. Je näher der User einem Knoten kommt, desto feingliedriger (langsamer) wird der Zoom-Schritt. Dies verhindert das nervige "Durch-Objekte-Hindurch-Clippen" im Mikrokosmos des Graphen.

07 Entwicklungs-Guide, Testing und Deployment

Dieses Kapitel richtet sich an Entwickler, die an der Codebase von Nodges (TypeScript, Vite, Three.js) mitarbeiten, und definiert die rigorosen Standards für Qualitätssicherung, Testing und den Deployment-Prozess.

1. Quality Assurance und Testing (Vitest)

Nodges nutzt modernste Pipelines und Validierungen (z.B. Zod), um Laufzeitfehler zu vermeiden. Dennoch sind Unit- und Integrationstests (besonders für den `LLMService` und `GraphDataManager`) unerlässlich. Als Framework kommt **Vitest** zum Einsatz.

1.1 Test-Suite und Commands

- `npm run test` â€“ Führt die gesamte Test-Suite im Watch-Modus aus. Ideal für TDD (Test-Driven Development).
- `npm run test:ui` â€“ Startet das Vitest UI-Dashboard im Browser. Dieses visuelle Tool ist extrem hilfreich, um komplexe Graphen-Datenstrukturen, die im Test fehlschlagen, zu debuggen.
- `npm run test:coverage` â€“ Generiert einen V8-basierten Coverage-Report (`coverage/`). Ziel für Kern-Logik-Klassen (insbesondere alles unter `src/utis/` und `src/managers/`) ist >80% Coverage.
- `npm run export:schema` â€“ Ein benutzerdefiniertes Skript (`vitest run src/tests/exportSchema.test.ts`), das die Zod-Schemas ausliest und als JSON-Schema für

die LLMs auf die Festplatte exportiert.

1.2 Testing-Muster

Bei der Entwicklung neuer Features in der Build-10-Pipeline müssen Mock-Responses geschrieben werden.

Das Testen von echten LLM-APIs in der CI/CD-Pipeline ist verboten (Flakiness & Kosten). Stattdessen wird der `LLMService` gemockt, um zu verifizieren, ob Nodges mit defektem JSON, fehlenden IDs in Kanten oder falschen Typen korrekt umgeht und diese über den ErrorHandler abfängt, ohne zu crashen.

2. Error Handling und Resilience

****Visualisierung:**** Siehe [10_ErrorHandlingResilience_002.mmd]
(10_ErrorHandlingResilience_002.mmd)

Nodges operiert oft an der Schnittstelle zu unzuverlässigen Drittsystemen (LLMs, Wikidata, Netzwerk). Das Error-Handling ist daher ein First-Class Citizen.

2.1 Der zentrale ErrorHandler

Jeder Catch-Block in der Applikation sollte an den zentralen ErrorHandler (oder ein Toast-System) delegieren.

- **Verbot von "White Screens":** Ein fehlerhafter API-Call oder ein missglücktes Zod-Parsing darf niemals dazu führen, dass die React/Vanilla-DOM-Schleife oder der Three.js-Render-Loop abstürzt.

- **Fail-Safe Mechanismen:**

- Erhält das System ein 429 Too Many Requests von OpenRouter, muss eine saubere Warnung ("Ratelimit erreicht, bitte lokales Modell nutzen") eingeblendet werden.
 - Wenn `App.loadGraphData` in der Mitte der Verarbeitung fehlschlägt, darf der `SceneManager` die alte (funktionierende) Szene nicht leeren. Der Zustand muss transaktionssicher (all-or-nothing) verarbeitet werden.
-

3. Lokale Entwicklungsumgebung

Der Tech-Stack von Nodges ist darauf ausgelegt, dass ein Entwickler das System in unter 2 Minuten lokal hochziehen kann.

1. **Abhängigkeiten:** `npm install` (Node.js >= 18 erforderlich).

2. **Dev-Server:** `npm run dev` startet den Vite-Server (meist auf `localhost:5173`) mit Hot-Module-Replacement (HMR). Änderungen an CSS oder TypeScript spiegeln sich in Millisekunden im Browser wider.

3. **Lokale LLM-Inferenz (Local-First):**

Für die Entwicklung an Prompts oder der Build-10-Pipeline ohne Token-Kosten wird der Einsatz von Ollama dringend empfohlen.

- Installiere Ollama lokal.

- Starte z.B. `ollama run llama3` oder `phi3`.

- Wähle in der Nodges-UI im CreatePanel den lokalen Provider aus. Das System feuert API-Calls dann an `http://localhost:11434/v1/chat/completions`.

4. Build und Deployment

****Visualisierung:**** Siehe [10_DevTestDeployPipeline_001.mmd]
(10_DevTestDeployPipeline_001.mmd)

Nodges ist eine "Frontend-only" bzw. "Static" Web-Applikation. Alle Berechnungen, LLM-Calls und API-Anfragen passieren direkt im Client (Browser). Es gibt kein proprietäres Node.js-Backend, das betrieben werden muss.

4.1 Produktions-Build

Der Befehl `npm run build` führt einen strikten TypeScript-Typecheck (`tsc`) aus und lässt Vite anschließend das Produktions-Bundle erstellen.

Das Ergebnis liegt im Ordner `/dist` . Die Dateien sind hochgradig minifiziert, CSS extrahiert und Three.js ge-tree-shaked (es wird nur ausgeliefert, was wirklich genutzt wird).

4.2 Hosting und BYOK (Bring Your Own Key)

- Das `/dist` Verzeichnis kann auf jedem statischen Webhoster abgelegt werden (GitHub Pages, Vercel, Netlify, Amazon S3).
- **BYOK-Architektur:** Da es kein Backend gibt, muss der User seine eigenen API-Keys in die Nodges-UI eintragen. Das System speichert diese Schlüssel strikt nur im lokalen `localStorage` des Browsers. Dies entbindet den Host von jeglicher DSGVO/Privacy-Haftung für sensible Daten-Prompts, da diese direkt vom Browser des Nutzers zum LLM-Provider wandern.

08 Historie und Entscheidungen (ADRs)

Das Entwicklungstagebuch von Nodges dokumentiert Architektur-Entscheidungen (Architecture Decision Records - ADRs) und Pivot-Momente. Es erklärt das "Warum" hinter den technischen Konzepten, um zu verhindern, dass zukünftige Iterationen alte, bereits gelöste Fehler wiederholen.

1. Die Findungsphase (Build 1 bis Build 4)

****Visualisierung:**** Siehe [11_BuildEvolutionsTimeline_001.mmd]
(11_BuildEvolutionsTimeline_001.mmd)

In den ersten Versionen war Nodges stark von klassischen Dashboards inspiriert.

- **Das Problem:** Die JSON-Daten waren hart an visuelle Komponenten gekoppelt. Wenn das System einen Knoten vom Typ "Server" fand, erwartete der Code ein zylindrisches Mesh; war es ein "Router", wurde ein Quader geladen.
 - **Der Pain Point:** Jeder neue Datensatz (z.B. Biologie, Mythologie) erforderte, dass der Quellcode angefasst wurde, um neue `if/else`-Blöcke für die Render-Logik zu schreiben. Die Skalierbarkeit war bei Null.
-

2. Der große Pivot: Build 5 (Die Entkopplung)

****Visualisierung:**** Siehe [11_OntologieArchitektKonzept_003.mmd]
(11_OntologieArchitektKonzept_003.mmd)

Build 5 ist der wichtigste Meilenstein in der Geschichte des Projekts.

- **Entscheidung:** Das Attribut `type` verlor seine Dominanz. Die Engine wurde ignorant gegenüber dem Inhalt.
- **Lösung:** Einführung von *Global Presets* (`global_node` , `global_edge`) und dem *Visual Mapping*. Datenattribute (wie `influence` , `category`) wurden nun zur Laufzeit durch den User auf visuelle Kanäle (Scale, Color) gemappt.
- **Geburt des Ontologie-Architekten:** Das LLM durfte nicht mehr einfach Listen von Objekten generieren. Es musste zuerst das Schema erfinden, welches dann das Mapping steuerte.

3. Der Kampf gegen Halluzinationen (Build 6)

****Visualisierung:**** Siehe [11_ADRProblemLoesung_002.mmd]
(11_ADRProblemLoesung_002.mmd)

Nachdem Build 5 die visuelle Flexibilität sicherstellte, fielen LLMs oft auf struktureller Ebene aus (sie brachen das JSON oder vergaßen Kanten-IDs).

- **Entscheidung:** Einführung strikter Laufzeit-Validierung.
- **Lösung:** Zod wurde in den Tech-Stack integriert. Das System nutzte nun den "JSON Schema Mode", um dem LLM einen harten Constraint aufzuerlegen. Das Modell konnte

physisch nicht mehr von der Datenstruktur abweichen.

4. Grounding und semantische Tiefe (Build 8 & Build 9)

Zwar generierten LLMs nun perfekte Strukturen, litten aber unter Wissenslücken bei harten Fakten (z.B. genaue Jahreszahlen, Entfernungen).

- **Build 8 (Wikidata):** Nodges delegierte die Recherche an Wikidata. Das LLM schrieb SPARQL-Queries statt Text zu raten. Die RDF-Triples wurden die verifizierte Basis für den 3D-Graphen. Halluzinationen bei harten Fakten wurden eliminiert.

- **Build 9 (Vektor-Deduplizierung):** Bei sehr großen Netzwerken wurden Duplikate generiert ("Apple" und "Apple Inc."). Build 9 führte Embeddings und Kosinus-Ähnlichkeit ein, um Graphen nach der Generierung semantisch zu verschmelzen und zu bereinigen.

5. Die Demokratisierung: Build 10 (Modulare Pipeline)

Die letzte große Architektur-Evolution. Früher gab es "die Build 6 Pipeline" oder "die Build 8 Pipeline" als getrennte Code-Pfade.

- **Entscheidung:** Monolithische Pipelines verhindern Flexibilität.

- **Lösung:** Build 10 wandelte Nodges in ein voll modulares, agentisches System um. Der Nutzer kombiniert im UI (CreatePanel) nun selbst: Modell-Quelle (Lokal vs. Cloud), Grounding-Strategie (Wikidata vs. None), QA (Human-in-the-loop vs. Auto) und

Bewertungsmethode (Cosine-Similarity). Dies machte Nodges zukunftssicher und unabhängig von einzelnen AI-Providern.

09 Quickstart und Tutorial

****Visualisierung:**** Siehe 12_QuickstartFlow_001.mmd

Dieses Tutorial führt Endbenutzer durch die ersten Schritte mit Nodges – von der Konfiguration der KI-Provider bis zur Navigation im 3D-Graphen.

1. Erstes Setup und Provider-Konfiguration

****Visualisierung:**** Siehe 12_ProviderSetupDetail_002.mmd

Nodges ist "Local-First" und "BYOK" (Bring Your Own Key). Das bedeutet, Sie müssen sich mit keinen proprietären Accounts anmelden.

1. **Einstellungen Öffnen:** Klicken Sie im UI auf das Zahnrad-Symbol (Settings).
2. **Provider wählen:**

- *Für Cloud:* Wählen Sie OpenRouter, OpenAI oder Anthropic. Tragen Sie Ihren persönlichen API-Key ein. Der Key wird sicher im `localStorage` Ihres Browsers gespeichert und verlässt Ihren Computer nur in Richtung des Providers.
 - *Für Lokal:* Wählen Sie "Local (Ollama)" oder "Local (LM Studio)". Stellen Sie sicher, dass Ihr lokaler Server (z.B. auf `http://localhost:11434`) läuft. Sie benötigen hierfür keinen API-Key.
-

2. Den ersten Graphen generieren (Build 10 Pipeline)

****Visualisierung:**** Siehe [12_PromptEngineering_006.mmd]
(12_PromptEngineering_006.mmd)

Die Magie von Nodges liegt in der agentischen Generierung.

1. Öffnen Sie das **Create Panel**.

2. Wählen Sie als Pipeline **"Build 10"**.

3. **Konfiguration:**

- Belassen Sie Grounding fürs Erste auf "Keines" (für maximale LLM-Kreativität).
- Setzen Sie QA auf "Generator + Kritiker" für hohe Datenqualität.

4. **Der Prompt:** Geben Sie im Textfeld ein Thema ein, z.B.: *"Zeichne mir das Ökosystem eines europäischen Waldes. Wer frisst wen? Wer lebt wo?"*

5. Klicken Sie auf **Generieren**. Lehnen Sie sich zurück, während die Toasts unten links Sie über die Phasen informieren (Ontologie wird erstellt -> Daten werden instanziiert).

3. Die Navigation im 3D-Raum

****Visualisierung:**** Siehe [12_KameraNavigationKontrollen_003.mmd]
(12_KameraNavigationKontrollen_003.mmd)

Sobald der Graph gerendert ist, nutzen Sie Maus oder Touchpad:

- **Rotieren (Orbit):** Linke Maustaste gedrückt halten und ziehen.
 - **Zoomen:** Mausrad (Scrollen) oder Pinch-to-Zoom auf Touchpads.
 - **Pannen (Verschieben):** Rechte Maustaste gedrückt halten und ziehen (oder Shift + Linke Maustaste).
 - **Interaktion:** Klicken Sie auf einen Knoten, um das Info-Panel an der Seite zu Öffnen und alle verborgenen Rohdaten (die `properties`) dieses spezifischen Knotens einzusehen.
-

4. Das Visual Mapping beherrschen

****Visualisierung:**** Siehe [12_VisualMappingMechanik_004.mmd]
(12_VisualMappingMechanik_004.mmd)

Der Graph sieht initial vielleicht noch unstrukturiert aus. Jetzt kommt die Decoupled UI ins Spiel. Öffnen Sie das **Mapping Panel**.

- **Größe Ändern:** Suchen Sie im Dropdown für `scale` nach numerischen Werten (z.B. `influence` , `weight`). Die Knoten skalieren sofort.
 - **Farbe Ändern:** Wählen Sie im Dropdown für `color` eine kategoriale Eigenschaft (z.B. `species` oder `habitat`). Nodges färbt die Cluster automatisch ein.
 - **Die Position sperren:** Das stärkste Tool. Wenn Sie im `position` Dropdown "Lebensdauer" auswählen und den Schalter auf die `y` -Achse legen, ordnen sich alle Knoten vertikal nach ihrem Alter an. Die Physik-Engine schaltet die Y-Achse ab, während X und Z weiterhin organisch floaten.
-

5. Eigene Dateien importieren (JSON)

****Visualisierung:**** Siehe [12_DragDropImportFlow_005.mmd]
(12_DragDropImportFlow_005.mmd)

Nodges ist nicht auf LLMs beschränkt. Sie können jederzeit standardisierte JSON-Dateien einlesen.

- Ziehen Sie einfach eine gültige `.json` Datei (welche dem Nodges Schema 5.0 entspricht) per Drag & Drop direkt in das Browserfenster. Das System validiert die Datei und rendert sie instantan.
-

10 Troubleshooting und FAQ

****Visualisierung:**** Siehe [13_TroubleshootingUebersicht_001.mmd]
(13_TroubleshootingUebersicht_001.mmd)

Dieses Dokument sammelt häufige Probleme (sowohl für Entwickler als auch für Endnutzer) und deren Lösungen, aufgeteilt in Backend/LLM, Frontend/WebGL und Architektur-Themen.

1. LLM und Generierungs-Probleme

Fehler: "JSON Parse Error" oder "Zod Validation Failed"

- **Symptom:** Der Generierungsprozess bricht mit einer roten Toast-Meldung ab; der alte Graph bleibt erhalten.
- **Ursache:** Das LLM hat (trotz JSON Schema Mode) halluziniert. Entweder hat es Text außerhalb des JSON-Blocks geschrieben, oder es hat Attribute eingefügt, die es in der Phase 1 (Ontologie) nicht definiert hatte. Auch fehlende IDs in Kanten sind ein häufiger Zod-Fehler.
- **Lösung:** Wenn Sie ein sehr kleines Modell (wie Llama-3-8B oder GPT-4o-mini) nutzen, switchen Sie auf ein potenteres Modell (GPT-4o, Claude 3.5 Sonnet). Kleine Modelle haben Probleme, komplexen Constraints über hunderte Zeilen JSON treu zu bleiben.

Fehler: CORS Error bei lokaler Ollama-Inferenz

****Visualisierung:**** Siehe [13_CORSArchitekturOllama_003.mmd]
(13_CORSArchitekturOllama_003.mmd)

- **Symptom:** Fetch-Error beim Versuch, das lokale Modell auf Port `11434` anzusprechen.
- **Ursache:** Ollama blockiert standardmäßig Cross-Origin-Requests aus dem Browser (`localhost:5173`).
- **Lösung:** Sie müssen Ollama mit aktivierten CORS-Headern starten. Unter Windows (PowerShell):

```
$env:OLLAMA_ORIGINS="*"; ollama serve (oder spezifisch http://localhost:5173 ).
```

Fehler: 429 Too Many Requests

- **Symptom:** Abbruch der Pipeline.
- **Ursache:** Rate-Limiting durch den Provider (z.B. OpenRouter Free-Tier) oder fehlendes Guthaben.
- **Lösung:** API-Key aufladen oder auf "Local" Provider umstellen.

2. WebGL, Engine und Performance

Symptom: Der Graph ruckelt stark (< 30 FPS)

- **Ursache 1:** Zu viele Kanten (Relationships). Das Rendern von tausenden Linien oder Zylindern belastet die GPU stark.
- **Ursache 2:** Die Physik-Engine läuft kontinuierlich auf der CPU.
- **Lösung:**
 - Reduzieren Sie im UI die "Force" oder setzen Sie "Cooldown", damit die Physik zur Ruhe kommt.
 - Nutzen Sie `InstancedMesh` für Geometrien (bereits Standard in der Engine).
 - Schalten Sie Antialiasing in den Settings ab, wenn Sie auf schwacher Hardware arbeiten.

Symptom: White Screen of Death beim Laden großer Graphen

****Visualisierung:**** Siehe [13_VRAMLeakKausalkette_001.mmd]
(13_VRAMLeakKausalkette_001.mmd)

- **Ursache:** Out-of-Memory (VRAM) Fehler der Grafikkarte. Alte Materialien wurden beim `clearScene` nicht korrekt disposed.
- **Lösung:** Prüfen Sie den `SceneManager.ts` daraufhin, ob alle Texturen, Geometrien und Arrays mit `.dispose()` behandelt wurden. Laden Sie die Seite mit `Ctrl + F5` neu.

3. Daten und Mapping

Symptom: Ein Mapping (Farbe/Größe) hat keine Auswirkungen

****Visualisierung:**** Siehe [13_OntologieInstanzDivergenz_002.mmd]
(13_OntologieInstanzDivergenz_002.mmd)

- **Ursache:** Die Eigenschaft (z.B. `influence`) existiert zwar in der Ontologie (`dataModel`), wurde aber vom LLM in den eigentlichen `entities` nicht befüllt (Null oder Undefined).
- **Lösung:** Klicken Sie auf einen der Knoten, um die Rohdaten einzusehen. Prüfen Sie, ob das Feld befüllt ist. Falls nicht, generieren Sie den Graphen neu und fordern Sie das LLM im Prompt explizit auf: *"Achte zwingend darauf, das Feld 'influence' bei jedem Knoten mit einem Wert zwischen 1 und 100 zu füllen."*

Symptom: Knoten fliegen chaotisch auseinander (Cluttering)

- **Ursache:** Die Repulsion-Force (Abstoßung) der Physik-Engine ist viel höher als die Link-Force (Anziehung der Kanten), oder der Graph ist unverbunden (Islands).
- **Lösung:** Justieren Sie die Physik-Slider im UI (Charge Strength verringern, Link Distance anpassen).

14 Ideen und Roadmap (Future Concepts)

****Visualisierung:**** Siehe [14_RoadmapFeatureHorizonte_001.mmd]
(14_RoadmapFeatureHorizonte_001.mmd)

Dieses Kapitel sammelt Konzepte, Evaluierungen und Architektur-Entwürfe, die zwar im Projektverlauf extrem detailliert ausgearbeitet, aber im aktuellen Build (10.x) **noch nicht oder nur teilweise implementiert** wurden. Sie dienen als Nordstern und Roadmap für künftige Erweiterungen.

1. Die Zeitdimension und Systemsimulation (TimeEngine)

****Visualisierung:**** Siehe [14_TimeEngineSimulation_002.mmd]
(14_TimeEngineSimulation_002.mmd)

Nodges ist aktuell primär ein statischer Viewer für Momentaufnahmen. Die größte anstehende Evolution ist die Transformation in einen lebendigen **Simulator**.

Das Konzept

- **Zeit-Parameter:** Knoten erhalten dynamische Verfallsraten (`decay`) und Schwellwerte (`thresholds`). Kanten erhalten Intervalle und Effekte.
- **Edge-Effekte:** Eine Kante (z.B. "verschmutzt") sendet Impulse in einem `interval` an den Zielknoten und reduziert dort beispielsweise einen Wert (z.B. "Gesundheit" via `operation: subtract`).
- **Visuelles Feedback:** Das System reagiert auf diese Veränderungen in Echtzeit. Sinkt der Wert eines Knotens, schrumpft er; fällt er unter einen Schwellwert, wechselt er die Farbe oder stirbt (Fade-Out).
- **Time Slider UI:** Ein dediziertes Premium-Panel am unteren Rand mit Play/Pause, Rewind und Geschwindigkeits-Reglern (Time-Warp), um den Lauf der Dinge (z.B. hunderte Jahre Evolution oder Millisekunden eines Server-Angriffs) zu steuern.

2. Geospatial Mapping (Karten-Integration)

****Visualisierung:**** Siehe [14_GeospatialMappingArchitektur_004.mmd]
(14_GeospatialMappingArchitektur_004.mmd)

Die Physik-Engine (Force-Directed) ist perfekt für abstrakte Topologien. Für Daten mit echtem geografischen Bezug (z.B. globale Lieferketten) greift jedoch ein anderes Konzept:

- **Das `metadata.map` Objekt:** Im JSON kann eine Referenzkarte (z.B. Weltkarte) hinterlegt werden.
- **Starre Koordinaten (`mapX` , `mapY`):** Knoten erhalten feste 2D-Koordinaten. Die Physik-Engine wird für diese Knoten deaktiviert, sie werden auf der Karte "festgepinnt".
- **3D-BÄumen:** Während die Knoten flach auf der Karte liegen, werden sich die Verbindungskanten (TubeGeometries) als parabolische Kurven im 3D-Raum darüber, um Verdeckungen zu vermeiden.

3. Externe Physik-Bibliothek zur Skalierung (`d3-force-3d`)

****Visualisierung:**** Siehe [14_BarnesHutSkalierung_003.mmd]
(14_BarnesHutSkalierung_003.mmd)

Aktuell berechnet ein nativer Web Worker die Abstoßung der Knoten mit einer $O(N^2)$ Komplexität.

- **Das Problem:** Bei über 2.000 Knoten führt dies zu Performance-Engpässen im Layout-Thread.
 - **Die Idee:** Die Implementierung der Bibliothek `d3-force-3d`. Diese nutzt den Barnes-Hut-Algorithmus (Octree), um die Berechnungs-Komplexität auf $O(N \log N)$ zu reduzieren, wodurch selbst 50.000 Knoten flüssig simuliert werden könnten, ohne dass eine Heavy-Weight Game-Engine (wie Cannon.js) benötigt wird.
-

4. Erweiterung der Beziehungsstrukturen (1-zu-n)

Aktuell unterstützt Nodges nur 1-zu-1 Kanten (`source` und `target`).

- **Die Idee:** Einführung eines `targets` -Arrays in den Edges. Im 3D-Raum würde dies als echtes sternförmiges Kraftmodell visualisiert (Eine dicke Hauptkante, die sich in mehrere kleine Kanten zu den Empfängern aufspaltet). Dies ist besonders für Broadcast-Systeme (z.B. "Server sendet an 50 Clients") essenziell.
-

5. RAG-Integration und Knowledge Graphs

Während Build 9 die Vektor-Deduplizierung einführte, geht die Vision weiter:

- **Die Idee:** Direkte Anbindung von Nodges an Vektordatenbanken (wie Pinecone oder ChromaDB).

- **Interaktion:** Der Nutzer klickt auf einen Knoten (z.B. "K nstliche Intelligenz") und dr ckt einen "Explore"-Button. Nodges feuert eine Vektorsuche ab, l dt die semantisch nahesten Konzepte live aus der Datenbank nach und instanziiert sie dynamisch im 3D-Raum ("Infinite Canvas").
-

6. Interoperabilit t (GEXF Export f r Gephi)

Eine weitreichende historische Evaluierung hat gezeigt, dass Nodges keine Insel sein darf.

- **Die Idee:** Eine Export-Funktion f r das XML-basierte **GEXF-Format**. Dies erlaubt es Akademikern und Datenanalysten, einen via LLM in Nodges erschaffenen Graphen zu exportieren, um ihn in wissenschaftlichen 2D-Werkzeugen (wie *Gephi*) mathematisch auf Eigenvector-Centrality oder Betweenness zu analysieren.
 - Ein bidirektionaler Import (Gephi -> Nodges) w rde zudem erm glichen, klassische, visuell unattraktive Graphen im Nodges-3D-System mit Glassmorphism- sthetik f r Pr sentationen aufzuwerten.
-

7. Modus-basiertes UI (Edit vs. View)

Aktuell verdecken Sidebar und Panels oft den majest tischen 3D-Graphen.

- **Die Idee:** Ein dezenter Toggle zwischen "Edit" und "View". Im View-Modus f hrt die komplette GUI weich aus dem Bildschirm und rollt sich zu minimalen Indikatoren am unteren Bildschirmrand zusammen. Das Resultat ist eine kinoreife, ablenkungsfreie Immersions-Pr sentation.

8. Unkonventionelle Kommunikations-Pfade

****Visualisierung:**** Siehe [14_NegativeDarstellung_005.mmd]
(14_NegativeDarstellung_005.mmd)

Die Limitierung auf optische Eindrücke ist nur der Anfang. Das Archiv beschreibt tiefergehende, sinnesübergreifende UX-Experimente:

- **Audio-visuelle Synopsie:** Knoten erzeugen beim Hovern eigene Klänge (tiefe Frequenzen für große Knoten, Dissonanz für "rote" Status-Knoten). Das Netzwerk wird hörbar.
- **Haptisches Feedback (VR/XR):** Vorbereitung für WebXR. Controller vibrieren beim Durchschneiden von High-Traffic-Kanten, Knoten bieten physischen "Widerstand" beim Greifen.
- **Fraktale Nodes:** Ein Knoten ist nicht nur ein Endpunkt. Beim extremen Heranzoomen entpuppt sich ein Knoten als komplettes eigenes Sub-Netzwerk (System-in-System Darstellung).
- **Negative Darstellung (Was fehlt?):** Das System invertieren. Anstatt "Was ist da?" zeigt Nodges die Phantome von zu erwartenden, aber fehlenden Verbindungen als gestrichelte, geisterhafte Linien (besonders wertvoll für Cybersecurity oder Supply-Chain Analyse).

15 Interoperabilität, Export und Multi-Build Formate

Nodges versteht sich nicht als Insel. Daten, die agentisch über LLMs generiert oder manuell strukturiert wurden, müssen portierbar bleiben. Dieses Kapitel beleuchtet die Interoperabilität mit externen Systemen und die Abwärtskompatibilität älterer Nodges-Datenformate.

1. Die Vision des GEXF-Exports (Graph Exchange XML Format)

****Visualisierung:**** Siehe [15_ExportImportFormate_001.mmd]
(15_ExportImportFormate_001.mmd)

Eine weitreichende historische Evaluierung (über 5 Konzept-Iterationen) hat das **GEXF-Format** als den idealen Brückenbauer zu klassischen, akademischen Netzwerkanalyse-Tools identifiziert.

Warum GEXF?

- **Gephi-Kompatibilität:** Gephi ist der Goldstandard für komplexe Netzwerkanalyse (z.B. die Berechnung von Eigenvector-Centrality, Betweenness, Modularity). Gephi rendert jedoch in 2D und ist visuell oft überladen.
- Nodges kann als agentischer "Datengenerator und 3D-Präsentator" dienen, während Gephi für die harte akademische Mathematik genutzt wird.

Die Architektur der GEXF-Übersetzung

- Das Nodges-JSON (mit seinen flachen `entities` und `relationships`) lässt sich verlustfrei in GEXF-Nodes und GEXF-Edges übersetzen.
 - **Continuous/Categorical Mappings:** Nodges-Attribute werden in `<attributes>` und `<attvalues>` Tags im GEXF-Format geparkt.
 - **Bidirektionaler Fluss:** Ein Export von Nodges nach Gephi und ein Re-Import des mathematisch angereicherten Graphen zurück nach Nodges (für die hochästhetische 3D-Glassmorphism-Präsentation) ist das Endziel der Interoperabilitäts-Schnittstelle.
-

2. Multi-Build Format Kompatibilität

****Visualisierung:**** Siehe [15_MultiBuildKompatibilitaet_002.mmd]
(15_MultiBuildKompatibilitaet_002.mmd)

Nodges entwickelt sich rasant weiter. Das JSON-Schema (aktuell Schema 5.0) verändert sich mit jedem Major-Release (Build 1 bis Build 10). Die Philosophie von Nodges ist jedoch: **Kein Format wird konvertiert, kein Format wird gelöscht.**

Parallele Schema-Verarbeitung

Nodges lädt verschiedene JSON-Schema-Versionen parallel und interpretiert diese korrekt, anstatt veraltete Dateien "kaputt-zu-migrieren":

- **Schema-Erkennung (DataParser):** Bevor der Datensatz durch Zod gejagt wird, liest der Parser die `schemaVersion` (oder analysiert die Knotenstruktur heuristisch).

- **Zod Union-Typen:** Das Zod-Schema in `types.ts` ist als `z.union` aufgebaut. Es akzeptiert sowohl Legacy-Builds (wo Attribute hart auf der Ebene von `node.properties` lagen) als auch moderne Builds (wo Daten in `stateVector` gekapselt oder flach gemappt sind).
 - **Abstraktionsschicht:** Eine Utility-Klasse (`BuildFormatUtils`) greift dynamisch auf die Daten zu, sodass die Render-Engine (Three.js) nicht wissen muss, ob sie ein Build-2 oder Build-10 JSON zeichnet.
-

3. Wissenschaftliche IE-Pipelines (Information Extraction)

****Visualisierung:**** Siehe [15_IEPipelineTripleExtraktion_003.mmd]
(15_IEPipelineTripleExtraktion_003.mmd)

Ein weiteres Konzept aus den Archiven betrifft den automatisierten Ingest von unstrukturierten Texten.

- Statt Nutzer-Prompts (wie im CreatePanel) sollen wissenschaftliche Paper per PDF-Upload in Nodges geparkt werden.
 - Eine dezidierte **IE-Pipeline** extrahiert strukturierte Triples (Subjekt-Prädikat-Objekt) und wandelt diese direkt in Nodges-Knoten und Kanten um, ohne dass der Nutzer das JSON-Schema jemals berühren muss.
-

16 Build 12: LightRAG Graph Retrieval Pipeline und Backend-Anbindung

Build 12 ist die fortschrittlichste Retrieval-Pipeline in Nodges. Sie ermöglicht die nahtlose Anbindung an lokale oder gehostete Knowledge-Bases über ein Python FastAPI-Backend.

1. Architektur und Backend-Struktur

Das Backend befindet sich im Ordner `lightrag-backend/` und nutzt `lightrag-hku` sowie `fastapi` und `uvicorn`.

- API-Endpunkte:

- `GET /health` : Liefert Server- und Engine-Status.
- `POST /query` : Verarbeitet RAG-Abfragen (`hybrid` , `local` , `global` , `naive`).
- `POST /insert` : Fügt Freitext oder Dokumente zur Wissensbasis hinzu.
- `GET /databases` : Formatiert und listet alle verfügbaren LightRAG-Datenbanken für das `FilePanelUI` .

- CORS & Environment:

Standardmäßig erlaubt das Backend Cross-Origin Requests vom Frontend (`http://localhost:5173`). API-Keys für OpenRouter und OpenAI Embeddings werden aus `.env.local` geladen.

2. Frontend Services und UI-Anbindung

Die Kommunikation erfolgt über `LightRAGService.ts`. Im `CreatePanel` steht Build 12 als eigene Pipeline ("Build 12: LightRAG") bereit.

- **Statusanzeige & Ingest:** Live-Erkennung des Backend-Status im CreatePanel sowie Buttons zum direkten Inserieren von Rohdaten.
 - **Relation Set Normalisierung:** Über den `DataParser.ts` werden zurückgegebene Entitäten und Beziehungen strukturiert auf das Schema 5.2 abgebildet.
-

3. Dokumentation & Referenz

Für eine umfassende technische Dokumentation von Build 12 siehe
[0_102_15_build12_lightrag_pipeline_dokumentation.md]
(0_102_15_build12_lightrag_pipeline_dokumentation.md).